3. *Less buggy code.* When you hand-code something complex, it's likely to have numerous bugs. Worse, with high complexity, you probably won't know what bugs there are in the hand-written code. The best way to overcome this problem is *not to write code at all !* The code is auto-generated from the specification, so if the specification is correct, you can be certain that the generated code is correct.

These advantages are well-known. That's why such auto-code generators appear to be a great idea. But that's not the end of the story. If you have considerable working experience with using such auto-code generators, you are probably aware of the numerous problems and disadvantages of using them. We'll discuss them in greater detail now.

Developers understand the code they have written, and will be able to understand code written by fellow programmers who are human beings. The generated code is auto-generated by a program, is often quite complex, and often unreadable—and it's not an exaggeration if I say it is sometimes a nightmare to read auto-generated code! So, maintainability and understanding suffers with auto-generated code.

The auto-generated code often needs to be customised to make it usable for your requirements or to get it working. For example, if you create screens using GUI builders, the generated code will contain empty event handlers with TODO entries for you to enter code for specific events such as mouse clicks and mouse moves. You'll have to figure out what these methods are, and understand what to write in there. They also contain hook methods for inserting your own logic or adding business logic. Often, developers add the bare minimum code necessary to get the program working. If the code segments with default behaviour are not exercised during the testing, the tests will pass and the software will get released. But bugs will be found in actual usage, and get filed as such. Forgetting to customise code is the cause of numerous bugs related to auto-generated code.

The major difficulty with auto-generated code is that it's difficult to integrate programmer code into the generated code. Consider the following scenario. You create large UML diagrams in a tool that can generate Java code. In the generated code, you have added lots of business logic, and made modifications to the parts of the generated code. For the next release of the software, you get new requirements and need to change the UML diagrams. However, you cannot make changes to the UML diagrams and generate code because the existing modified code is not in sync with the newly generated code! This occurs, assuming that the UML design tool does not support syncing the changes back to the diagrams from the code, which is the case with most of the tools available today. Hence, you need to make all the changes in the actual code from the earlier version, without using the UML design tool, which defeats the very purpose of auto-generating code—this is the most significant problem with it.

One practical way to integrate generated code with the programmer's code is to maintain strict separation between them by keeping them in physically separate files. In this approach, programmers don't directly make changes to the file containing generated code. Rather, they make changes in a separate file, for example, by overriding the hook methods. So, an argument to this approach is that if the higher-level specification or model changes, then the code can be regenerated, and that will not impact the programmer-written code. Yes, this solves some of the problems with mixing generated code and programmer written code. However, it still does not address the problem of when the generated code needs to be modified, particularly in the context of supporting Non-Functional Requirements (NFRs).

Consider, for example, that you want to improve the performance of your software. Generated code, because it is generic and has numerous hook methods, is often inefficient. Similarly, consider that you want to make your code thread-safe, and assume that the auto-generated code is not. In both these cases, you cannot directly make modifications to the generated code, because if you regenerate the code from the generator tool, those changes will be lost. So, separating generated code from programmers' code solves some problems, but not all.

In practice, the evolution of the auto-generator tool can also cause problems. Assume that you're working on a maintenance project that made use of auto-generated code. Now you want to regenerate the code with new modifications, but you have access only to the new version of the auto-generator tool. This new version generates code that is incompatible with the code generated by the older version of the tool; so if you use the code generated from the new version of the tool, your existing code (written to be compatible with the code in the older format) will not work. You'll then be forced to modify the generated code by hand, and not use the newer version of the tool.

There are other problems as well that I don't delve into too deeply, but I leave it to you to think about. Assuming that you use auto generators in your project, and face these problems, how will you handle the following situations:

- There are many cases where the generated code has bugs, which means that the generator tool generates a buggy program (this is not an uncommon problem).
- You've coding guidelines for your project that need to